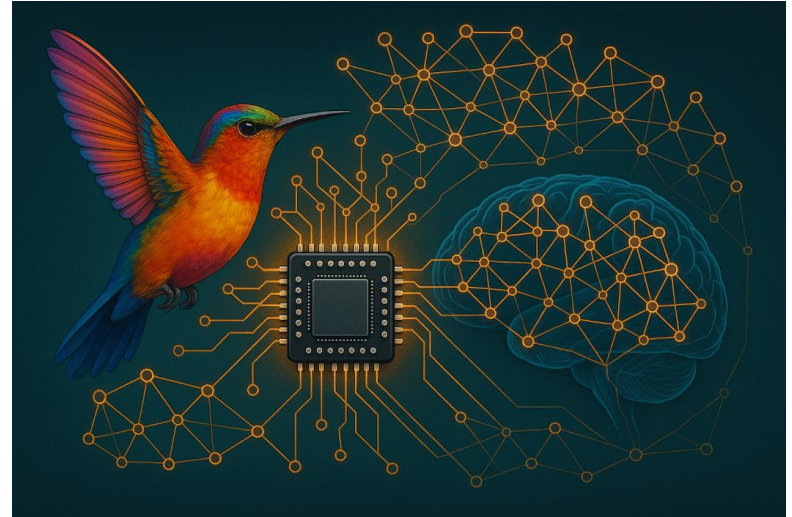


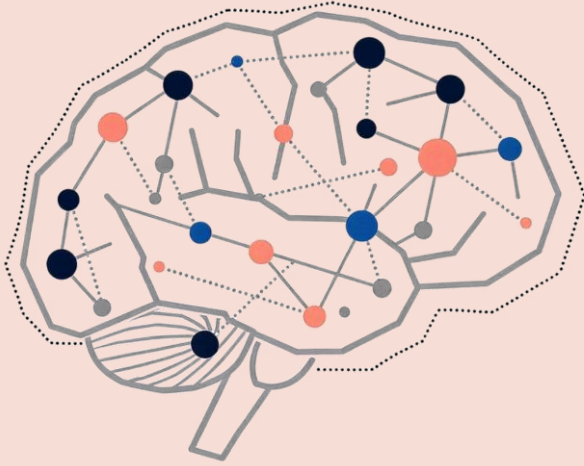
El paradigma del Machine Learning



Fuente: ilustración generada con IA (ChatGPT, OpenAI). Ernesto Sifuentes, 19 ago. 2025.

Inteligencia artificial embebida: sistemas electrónicos con capacidades cognitivas.

Prof. Ernesto Sifuentes de la Hoya
UACJ – Universidad Autónoma de Ciudad Juárez, Chih., México
IIT – Instituto de Ingeniería y Tecnología
Dpto. Ingeniería Eléctrica y Computación



El paradigma del Machine Learning

De la programación explícita al aprendizaje automático

LAURENCE MORONEY, GOOGLE

Introducción



PROGRAMACIÓN CLÁSICA (BASADA EN REGLAS)

TÚ ESCRIBES LAS REGLAS



REGLAS ESCRITAS

SI fiebre > 38°C
Y dolor de cabeza
ENTONCES gripe

SI tos
Y congestión
ENTONCES resfrío

...

SALIDA

El sistema
aplica las reglas
y da una
respuesta.



El programador debe anticipar y escribir
todas las situaciones posibles.

VS.

MACHINE LEARNING

LE DAS EJEMPLOS AL SISTEMA
Y EL SISTEMA APRENDE

EJEMPLOS (DATOS)

Fiebre	Tos	Dolor	Diagnóstico
✓	✓	✓	Gripe
✓	✓	✗	Resfrío
✗	✓	✓	Alergia
...	...	...	...

SISTEMA



El sistema ajusta
sus parámetros
y encuentra patrones.

NUEVOS CASOS

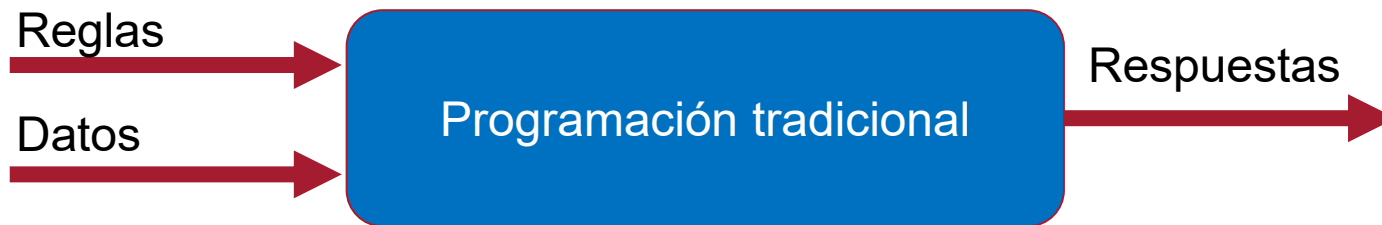
Fiebre ✓
Tos ✓
Dolor ✗

PREDICCIÓN

Probable: Gripe

El sistema *aprende* de los datos
y *mejora* con más ejemplos.

Programación explícita



Ejemplos:

Filtro de spam clásico

if email.contains('oferta') or
email.contains('gratis') → SPAM

Termostato inteligente

if temperatura > 22°C →
apagar calefacción

Detector de fraude bancario

if monto > 10,000 AND país != origen
→ bloquear

Programación explícita

Reglas predefinidas

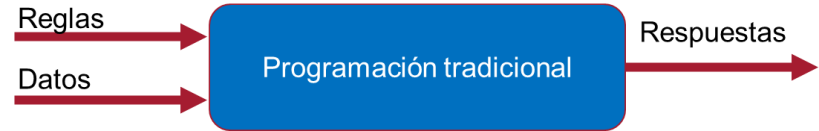
El programador define cada comportamiento posible explícitamente.

Todo precalculado

Nada ocurre fuera de lo que el código prevé.

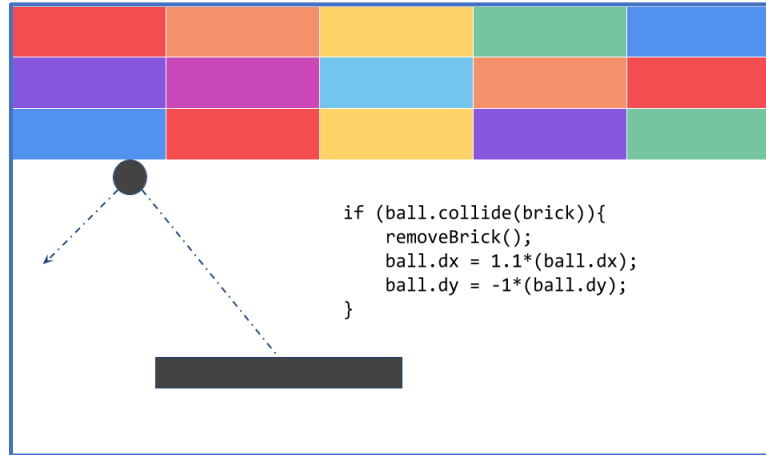
Escenarios limitados

La complejidad del programa define los límites del sistema.



Problema: ¿qué pasa cuando las reglas son miles?
¿O cuando el entorno cambia?
El código se vuelve imposible de mantener.

El paradigma tradicional



✓ Funciona bien cuando...

- ❖ Las reglas son pocas y claras.
- ❖ El problema no cambia con el tiempo.
- ❖ Los casos de borde son predecibles.

✗ Falla cuando...

- ❖ Las reglas son demasiadas o complejas.
- ❖ El entorno cambia constantemente.
- ❖ Hay patrones difíciles de articular en código.

Detección de actividad



Caso simple: caminar

Una sola condición es suficiente para detectar la actividad.

```
if(speed < 4) { status = WALKING; }
```

```
if(speed<4){  
    status=WALKING;  
}
```

Fácil cuando solo existe una actividad posible.

Detección de actividad



```
if(speed<4){  
    status=WALKING;  
} else {  
    status=RUNNING;  
}
```

Ahora también: **correr**

El código crece para cubrir el nuevo caso.

```
if(speed < 4) { status = WALKING; }  
  
else { status = RUNNING; }
```

Cada actividad nueva requiere una regla nueva.

Detección de actividad



Añadimos: ciclismo

```
if(speed<4){  
    status=WALKING;  
} else if(speed<12){  
    status=RUNNING;  
} else {  
    status=BIKING;  
}
```

```
if(speed < 4) { status = WALKING; }  
else if(speed < 12) { status = RUNNING;  
} else { status = BIKING; }
```

El código se vuelve más frágil con cada nueva actividad.

Actividades más complejas



// ???

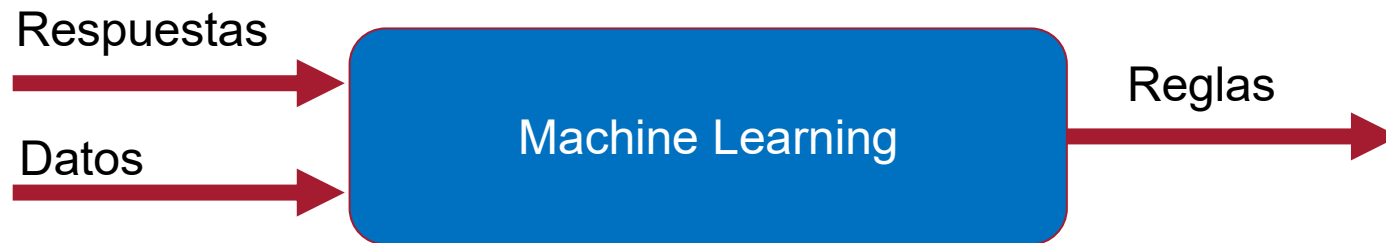
El problema escala

Golf, natación, escalada, yoga... cada nueva actividad exige más reglas, más condiciones y más errores potenciales.



Las reglas escritas a mano no escalan. El código se vuelve inmanejable.

El paradigma del Machine Learning



En ML, se invierte el proceso: se proporcionan los **datos** y las **respuestas**, y el modelo aprende las **reglas** por sí mismo.

Detección de actividad con ML

- ❖ En lugar de programar reglas para cada actividad, entrenamos un modelo con ejemplos etiquetados.
- ❖ El modelo descubre los patrones automáticamente.



```
0101001010100101010
1001010101001011101
0100101010010101001
0101001010100101010
```

Label = WALKING



```
1010100101001010101
0101010010010010001
0010011111010101111
1010100100111101011
```

Label = RUNNING



```
1001010011111010101
1101010111010101110
1010101111010101011
1111110001111010101
```

Label = BIKING



```
1111111111010011101
0011111010111110101
0101110101010101110
1010101010100111110
```

Label = GOLFING

Detección de actividad con ML

			
<pre> 0101001010100101010 1001010101001011101 0100101010010101001 0101001010100101010 </pre>	<pre> 1010100101001010101 0101010010010010001 0010011111010101111 1010100100111101011 </pre>	<pre> 1001010011111010101 1101010111010101110 1010101111010101011 1111110001111010101 </pre>	<pre> 1111111111010011101 001111101011110101 0101110101010101110 10101010100111110 </pre>
Label = WALKING	Label = RUNNING	Label = BIKING	Label = GOLFING

Datos etiquetados

Ejemplos de actividades con su etiqueta correcta.

Entrenamiento ML

El modelo descubre patrones Automáticamente.

Modelo

Detecta cualquier actividad sin reglas manuales.

Detección de actividad con ML

Lo que el modelo aprende

Caminar

Patrones de velocidad y aceleración bajos.

Correr

Velocidad alta, aceleración rítmica.

Ciclismo

Velocidad sostenida, aceleración suave.

Golf / Yoga

Movimientos lentos, patrones de postura.

Ventaja sobre reglas manuales

- ❖ Una sola arquitectura detecta todas las actividades.
- ❖ Añadir nuevas actividades = más datos, no más código.
- ❖ El modelo descubre patrones que el humano no vería.
- ❖ Funciona con sensores imperfectos o ruidosos.

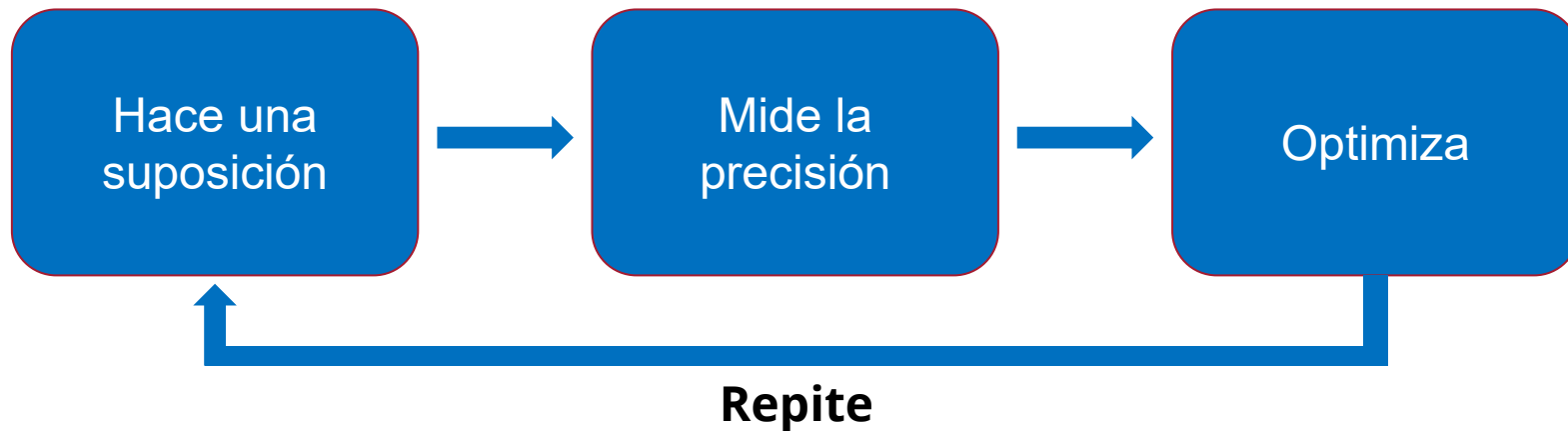


Con ML, pasar de 3 a 10 actividades no requiere reescribir el código — solo más datos etiquetados.

¿Cómo aprende el modelo?



El aprendizaje ocurre mediante un ciclo iterativo de **estimaciones** y **correcciones**.



Suposición inicial

El modelo genera una predicción con parámetros aleatorios.

Medición del error

Se calcula la diferencia entre predicción y valor real.

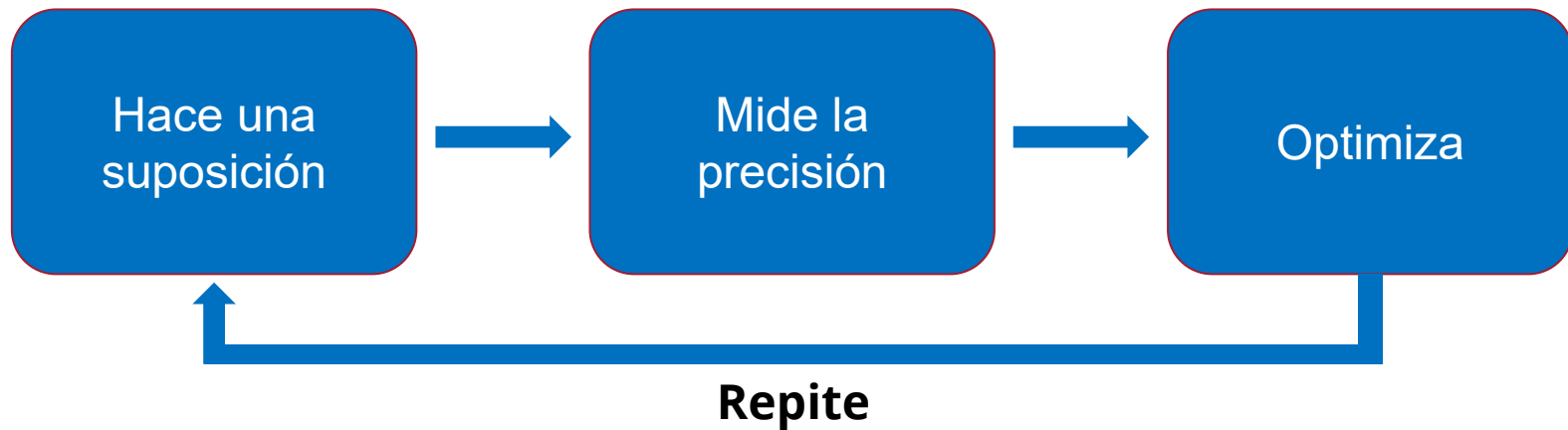
Corrección

Los parámetros se ajustan para reducir el error.

¿Cómo aprende el modelo?



El aprendizaje ocurre mediante un ciclo iterativo de **estimaciones** y **correcciones**.



Este ciclo se repite cientos o miles de veces hasta que el modelo converge.

¿Cómo aprende el modelo?



Cocina y Machine Learning

Una analogía simple para entender cómo aprende un modelo



Así como una receta se ajusta al probar la sopa,
un modelo ajusta sus pesos al medir el error.



Aprender consiste en **probar**, **medir el error**, **ajustar** y **repetir** hasta **converger**.

Fuente: ilustración generada con IA (ChatGPT, OpenAI). Ernesto Sifuentes, 15 mayo de 2026.

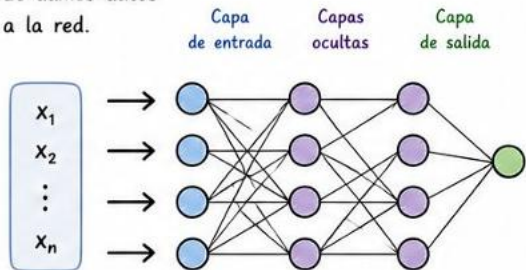
¿Cómo aprende el modelo?



Una red neuronal aprende haciendo, comparando y ajustando.
Así mejora sus predicciones con el tiempo.

1 Entrada

Le damos datos a la red.



2 Predicción

La red hace una predicción.

Salida predicha
 $\hat{y}$

3 Comparación

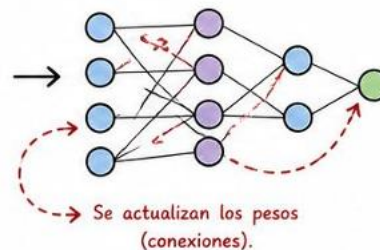
Comparamos con la respuesta correcta.

Respuesta correcta
 y

Error
 $(\hat{y} - y)$

4 Ajuste

La red ajusta sus conexiones internas para reducir el error.



5 Repetición

El proceso se repite muchas veces hasta que el error es muy pequeño.



El paradigma ML: pasos 1–3

1



Hace una suposición

El modelo genera una predicción $\hat{Y}$ con parámetros actuales w .



2



Mide la precisión

Calcula la pérdida:

$$L = MSE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



3



Optimiza

Actualiza pesos:

$$w_{t+1} = w_t - \alpha \cdot \frac{\partial L}{\partial w}$$

para reducir el error.

Ejemplo concreto

Suposición

Error (MSE)

$Y = 3X - 1$ (inicial)

5.57 ✗

$Y = 2X - 2$ (iteración 2)

2.45 ↻

$Y = 2X - 1$ (final)

0.00 ✓



En solo 3 iteraciones el modelo encontró la fórmula exacta. En la práctica, las redes neuronales lo hacen con millones de parámetros.

El ciclo de aprendizaje

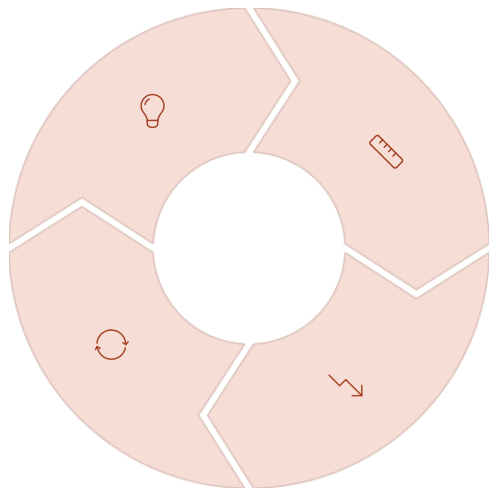


Suposición

El modelo predice con parámetros actuales

Actualización

$w \leftarrow w - \alpha \cdot \nabla L$, repetir



Medición

La función de pérdida cuantifica el error

Optimización

El gradiente indica cómo ajustar los pesos

¿Cuántas veces se repite?

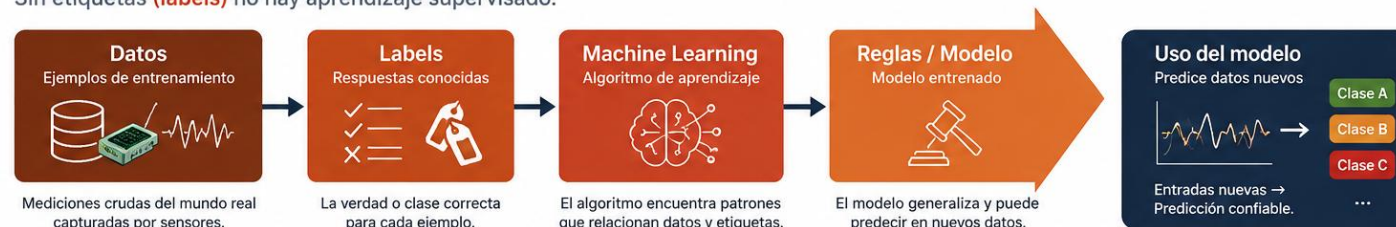
- ❖ Cada repetición = 1 época.
- ❖ Típicamente 100 a 10,000 épocas.
- ❖ Depende de la complejidad del problema.
- ❖ Early stopping evita sobreajuste.

¿Qué mejora en cada ciclo?

- ❖ Los pesos w se acercan a los valores óptimos.
- ❖ La pérdida disminuye progresivamente.
- ❖ Las predicciones se vuelven más precisas.
- ❖ El gradiente se aproxima a cero..

Labels: el ingrediente clave

Sin etiquetas (**labels**) no hay aprendizaje supervisado.



✓ Las **etiquetas (labels)** son las respuestas conocidas que usamos para **entrenar al modelo**. Sin ellas, no hay aprendizaje supervisado.

EJEMPLOS DE DATOS ETIQUETADOS (con sensores en ESP32-S3)

Tipo de problema	Sensores usados (ejemplo)	Ejemplo de datos (ventana de tiempo)	Etiqueta (label)	Notas clave															
Clasificación (Actividades)	Acelerómetro + Giroscopio (IMU)	<table> <tr><td>0.12</td><td>0.14</td><td>0.10</td><td>...</td><td>0.08</td></tr> <tr><td>-0.05</td><td>0.02</td><td>-0.03</td><td>...</td><td>0.01</td></tr> <tr><td>9.81</td><td>9.78</td><td>9.77</td><td>...</td><td>9.80</td></tr> </table>	0.12	0.14	0.10	...	0.08	-0.05	0.02	-0.03	...	0.01	9.81	9.78	9.77	...	9.80	Caminar / Correr / Sentado / De pie / Subir escaleras	El modelo aprende patrones temporales en señales IMU.
0.12	0.14	0.10	...	0.08															
-0.05	0.02	-0.03	...	0.01															
9.81	9.78	9.77	...	9.80															
Clasificación (Estados de salud)	PPG (ritmo cardiaco) + Temperatura	<table> <tr><td>68</td><td>69</td><td>71</td><td>...</td><td>70</td></tr> <tr><td>36.4</td><td>36.4</td><td>36.5</td><td>...</td><td>36.6</td></tr> </table>	68	69	71	...	70	36.4	36.4	36.5	...	36.6	Normal / Estrés / Fatiga / Ejercicio	Las etiquetas vienen de expertos o mediciones de referencia.					
68	69	71	...	70															
36.4	36.4	36.5	...	36.6															
Clasificación (Objetos magnéticos)	Sensor magnético (Hall) (A3144 / QMC5883L)	<table> <tr><td>120</td><td>118</td><td>121</td><td>...</td><td>119</td></tr> <tr><td>-15</td><td>-18</td><td>-16</td><td>...</td><td>-14</td></tr> <tr><td>35</td><td>34</td><td>36</td><td>...</td><td>37</td></tr> </table>	120	118	121	...	119	-15	-18	-16	...	-14	35	34	36	...	37	Imán cerca / Imán lejos / Sin imán	Ideal para detección de proximidad o identificación.
120	118	121	...	119															
-15	-18	-16	...	-14															
35	34	36	...	37															
Regresión (Temperatura)	Sensor interno de Temperatura (TSENS)	<table> <tr><td>31.2</td><td>31.3</td><td>31.1</td><td>...</td><td>31.0</td></tr> </table>	31.2	31.3	31.1	...	31.0	Valor continuo (°C) (Regresión)	Predice un valor numérico continuo.										
31.2	31.3	31.1	...	31.0															
Clasificación (Audio)	Micrófono (I2S digital)		Palabra: "Hola" / "Adiós" / "Sí" / "No"	Requiere extracción de características (MFCC, etc.).															
Clasificación (Gestos)	IMU (acel + giro) + Procesamiento temporal	<table> <tr><td>0.03</td><td>0.04</td><td>0.02</td><td>...</td><td>-0.01</td></tr> <tr><td>-0.01</td><td>-0.02</td><td>0.00</td><td>...</td><td>0.02</td></tr> <tr><td>0.02</td><td>0.03</td><td>0.01</td><td>...</td><td>-0.03</td></tr> </table>	0.03	0.04	0.02	...	-0.01	-0.01	-0.02	0.00	...	0.02	0.02	0.03	0.01	...	-0.03	Gesto: Arriba / Abajo / Izquierda / Derecha / Flick / Rotación	Excelente para interfaces naturales e IoT.
0.03	0.04	0.02	...	-0.01															
-0.01	-0.02	0.00	...	0.02															
0.02	0.03	0.01	...	-0.03															

BUENAS PRÁCTICAS PARA ETIQUETAR



Consistencia
Usa criterios claros y consistentes para todas las muestras.



Cantidad y variedad
Reúne muchos ejemplos de diferentes personas, entornos y condiciones.



Calidad > Cantidad
Etiquetas correctas valen más que miles de etiquetas errores.



Documenta todo
Guarda metadatos: quién, cuándo, cómo y en qué condiciones.



Balance de clases
Evita clases muy desbalanceadas.

Del entrenamiento a la inferencia



Fase de entrenamiento

↓
Labels + Datos
Ejemplos etiquetados de entrenamiento

↓
Machine Learning
Algoritmo de optimización

↓
Reglas (Modelo)
Parámetros aprendidos

Fase de inferencia

↓
Nuevos datos
Ejemplos sin etiquetar

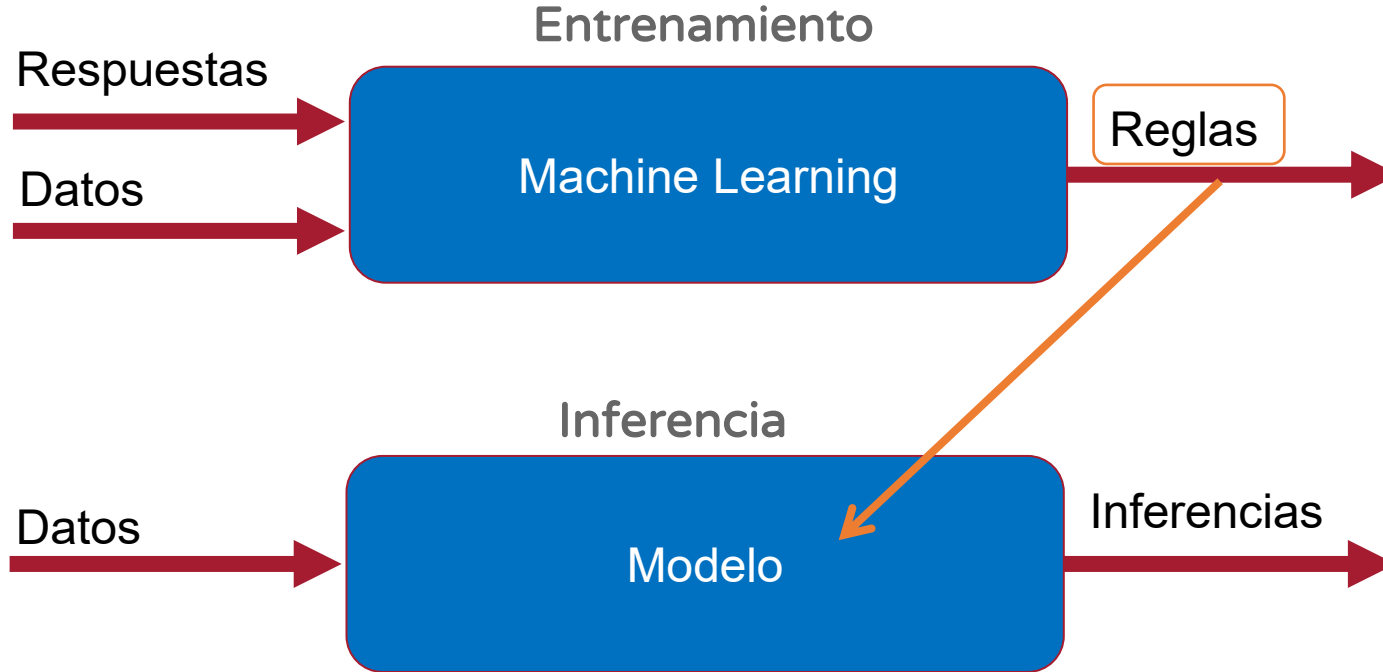
↓
Modelo
Aplica las reglas aprendidas

↓
Predicciones
Inferencias sobre nuevos datos



El modelo entrenado se convierte en la "**regla**" que antes escribíamos a mano.

El paradigma del Machine Learning



Problema: encontrar la relación entre X y Y

X	Y
-1	-3
0	-1
1	1
2	3
3	5
4	7

¿Cuál es el patrón?

- ❖ Cada vez que X aumenta , Y aumenta
- ❖ Cuando $X=0$, $Y=-1$
- ❖ La relación es lineal

La fórmula oculta

$$Y = 2X - 1$$

El modelo aprenderá esta fórmula automáticamente a partir de los datos, sin que nadie se la diga.

¡Hace una suposición!

X	-1	0	1	2	3	4
Y predicho ($Y = 3X - 1$)	-4	-1	2	5	8	11
Y real	-3	-1	1	3	5	7
Diferencia	-1	0	1	2	3	4

Diferencias ($\hat{Y} - Y_{\text{real}}$)

El problema



Las diferencias positivas y negativas se cancelan al sumar $\rightarrow$ necesitamos otra métrica.

La solución: elevar al cuadrado

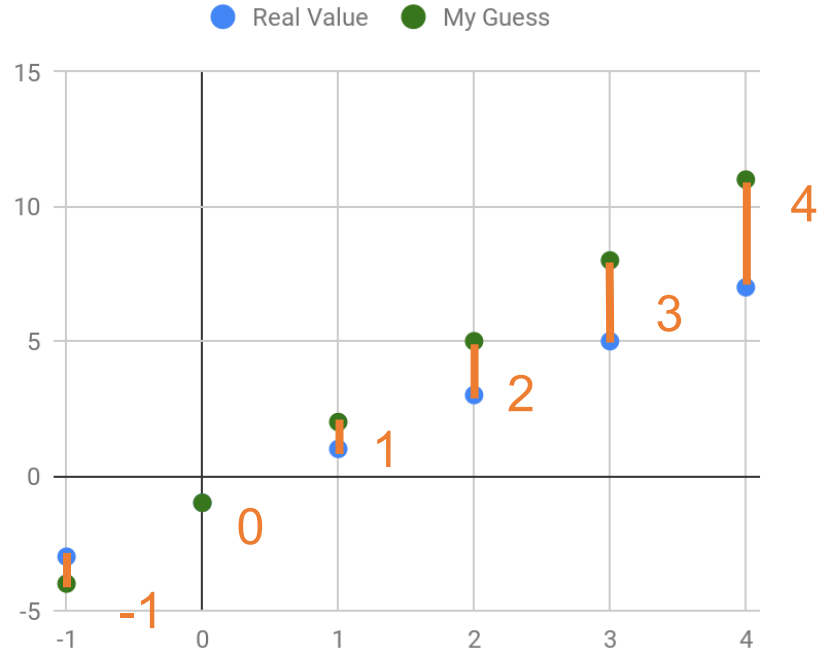


Diff²: 1, 0, 1, 4, 9, 16 $\rightarrow$ Suma = 31
 $\rightarrow \sqrt{31} \approx \mathbf{5.57}$ (error total)

¡Hace una suposición!



Diferencias ($\hat{Y} - Y_{\text{real}}$)



¡Estima el error!

Se requiere una función que cuantifique qué tan lejos estamos de la respuesta correcta.

¿Qué se compara?

$\hat{Y}$ — Predicción del modelo

Lo que el modelo cree que es la respuesta correcta.

Y — Valor real (label)

La respuesta correcta del conjunto de entrenamiento.

Tipos de error

MAE

Error Absoluto Medio: promedio de $|Y - \hat{Y}|$.
Fácil de interpretar.

MSE

Error Cuadrático Medio: promedio de $(Y - \hat{Y})^2$.
Penaliza errores grandes.

RMSE

Raíz del MSE: mismas unidades que Y . El más intuitivo.

Error = diferencia entre lo que predijo el modelo y lo que era correcto

 Para nuestro ejemplo, $Y = 2X - 1$, usamos el MSE. Con $Y = 3X - 1$ como primera suposición, el error fue $\sqrt{31} \approx 5.57$.

Nueva suposición: $Y = 2X - 2$

Segunda iteración: ajustamos los parámetros

X	-1	0	1	2	3	4
Mi Y	-4	-2	0	2	4	6
Y real	-3	-1	1	3	5	7
Diff²	1	1	1	1	1	1



Error = $\sqrt{(1+1+1+1+1+1)} = \sqrt{6} \approx \mathbf{2.45}$ — ¡Mejor que **5.57**!

Mismo proceso, mejor resultado

Suposición anterior: $Y = 3X - 1$ Nueva suposición: $Y = 2X - 2$

5.57

2.23

$$\sqrt{1 + 1 + 1 + 1 + 1} = \sqrt{5} \approx 2.23$$

El error bajó de 5.57 a 2.23. ¡Estamos mejorando! Repetimos el proceso.

¡Suposición perfecta! $Y = 2X - 1$

Tercera iteración: convergencia total.

X	-1	0	1	2	3	4
Mi Y	-3	-1	1	3	5	7
Y real	-3	-1	1	3	5	7
Diff²	0	0	0	0	0	0



Error = 0. El modelo ha aprendido la función exacta: $Y = 2X - 1$

Minimizando la pérdida...

El objetivo

Encontrar los parámetros w que minimizan la función de pérdida $L(w)$

$$\min_w L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

La estrategia

1

Calcular pérdida

¿Qué tan lejos estamos?

2

Calcular gradiente

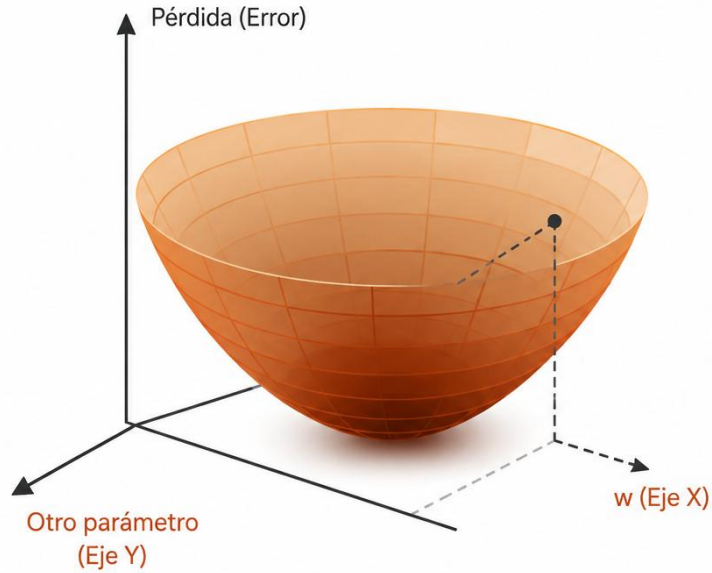
¿En qué dirección bajar?

3

Actualizar pesos

Dar un paso hacia el mínimo

La función de pérdida



Eje X:

valor del parámetro w



Eje Y:

error del modelo



El valle más profundo

= mejor modelo



La función de pérdida mapea los parámetros del modelo contra el error.
Nuestro objetivo es encontrar el **mínimo**.

La función de pérdida

Función de
pérdida

- ❖ Moverse en la dirección del gradiente.
- ❖ La tasa de aprendizaje es el tamaño del paso a dar.

$$L = MSE(Y, \hat{Y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

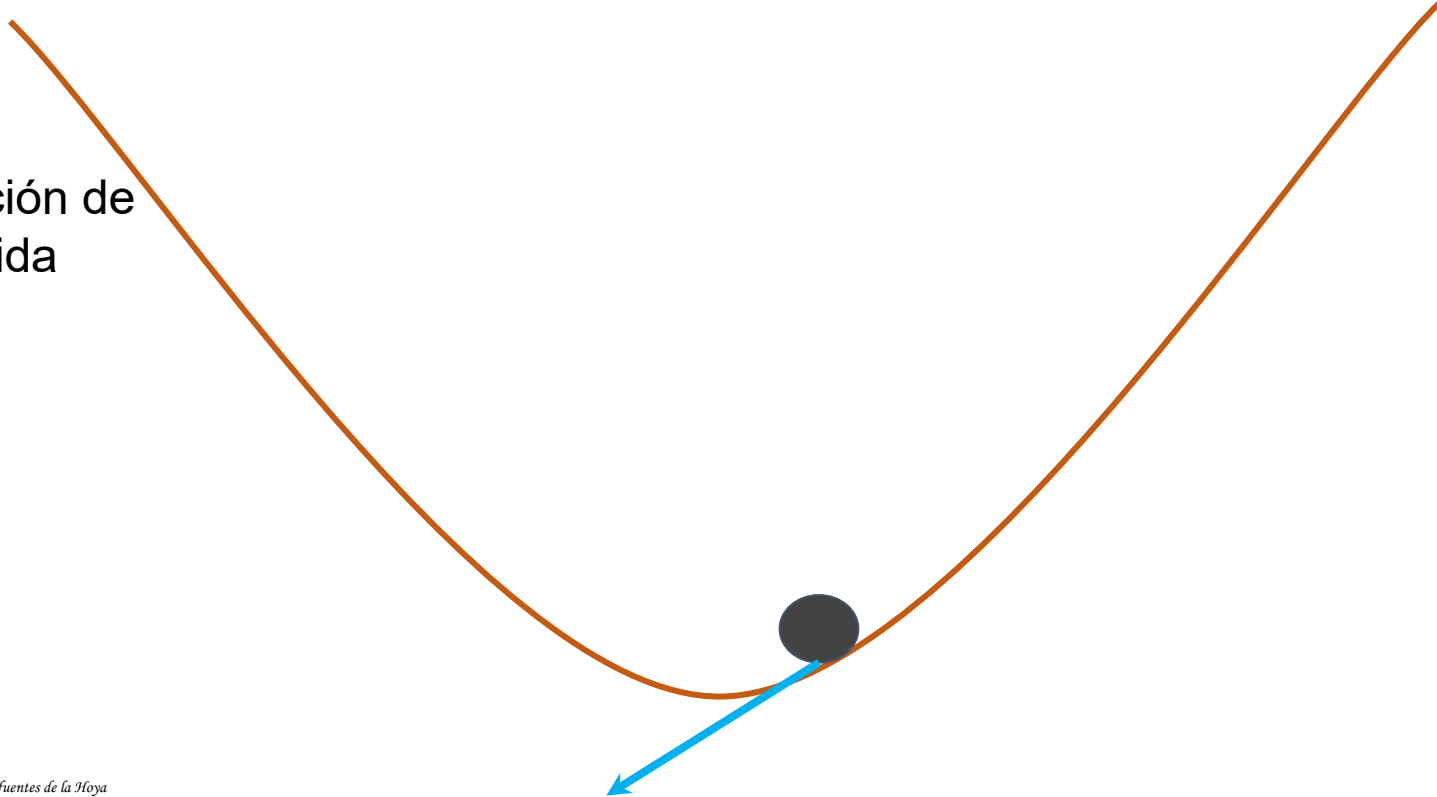
$$\min_w L(w) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$w_{t+1} = w_t - \alpha \cdot \frac{\partial L}{\partial w}$$

La función de pérdida



Función de
pérdida





El mínimo de la función de pérdida

INICIO

Pérdida alta

Parámetros aleatorios. El modelo predice mal.

PROCESO

Pérdida en descenso

El gradiente guía los ajustes iteración a iteración.

OBJETIVO

Pérdida mínima ✓

Parámetros óptimos. Las predicciones son precisas.



En el mínimo, el error es mínimo y las predicciones son óptimas.

El gradiente da la dirección

$$\nabla L = \frac{\partial L}{\partial w}$$

Gradiente positivo (+)

La pérdida sube al aumentar $w \rightarrow$ moverse a la izquierda (decrementar w).

Gradiente = 0

Estamos en el mínimo $\rightarrow$ no hay que moverse.

Gradiente negativo (-)

La pérdida baja al aumentar $w \rightarrow$ moverse a la derecha (incrementar w).



Siempre nos movemos en la dirección opuesta al gradiente para descender.

Tasa de aprendizaje



$$w_{\{t+1\}} = w_t - \alpha \nabla L \quad \rightarrow \quad w_{\{t+1\}} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

α muy pequeña

Pasos diminutos. Converge pero requiere miles de épocas. Lento.

α óptima

Pasos equilibrados. Converge eficientemente al mínimo global.

α muy grande

Pasos gigantes. Salta el mínimo. Puede divergir y nunca converger.

Valores típicos: $\alpha = 0.001$ a 0.01 . Se ajusta mediante técnicas como el learning rate scheduling.

Mínimos locales vs. globales

Mínimo local ⚠️

Qué es	El problema
Un valle pequeño, no el más profundo de la curva de pérdida .	Gradiente = 0, el algoritmo puede quedar atrapado .

Mínimo global ✅

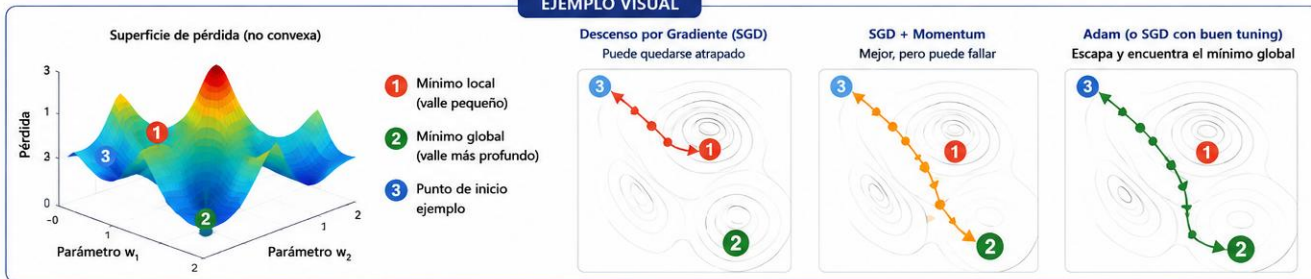
Qué es	La solución
El punto más bajo de toda la función de pérdida .	Adam y SGD con momentum escapan de mínimos locales.



¿QUÉ SIGNIFICA ESTO?

En funciones convexas, cualquier punto de inicio converge al mismo mínimo global.
En redes profundas, los optimizadores modernos manejan la no-convexidad.

EJEMPLO VISUAL



En redes neuronales profundas la **función de pérdida no es convexa** — los mínimos locales son un desafío real.



Buena inicialización
Puede ayudar a estar más cerca de buenas regiones.



Datos y etiquetas de calidad
Mejoran el paisaje de pérdida y la generalización.



Optimizadores modernos
Adam, RMSProp, y SGD + momentum manejan la no-convexidad.

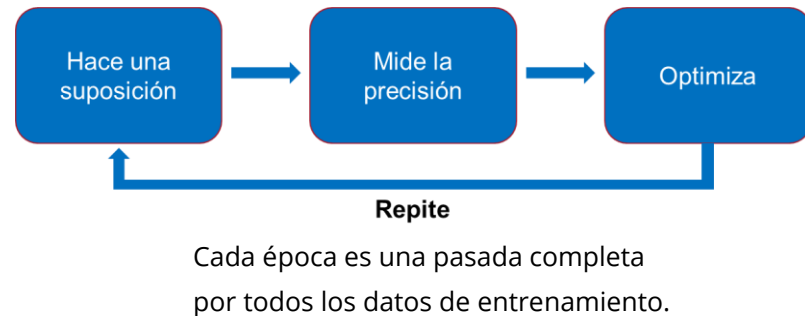
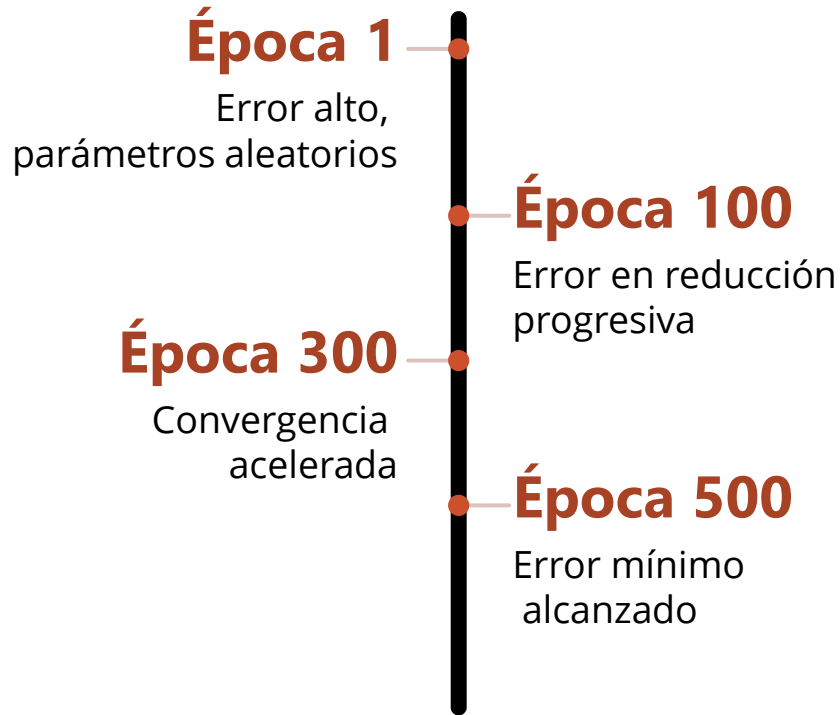


Tasa de aprendizaje adecuada
Muy alta: inestable. Muy baja: muy lento o se atasca.



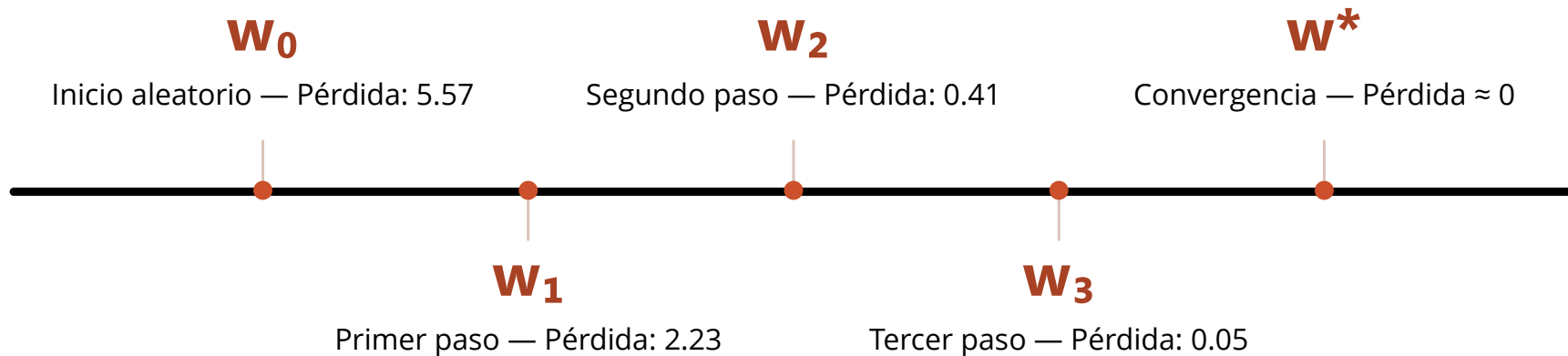
Regularización
Evita sobreajuste y mejora la superficie de pérdida.

Épocas de entrenamiento



Moviéndose hacia el mínimo

$$w_{\{t+1\}} = w_t - \alpha \frac{\partial L}{\partial w_t}$$



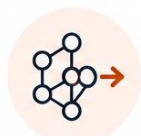
En cada iteración

- ❖ Calcular gradiente ∇L en posición actual
- ❖ Dar paso: $w \leftarrow w - \alpha \cdot \nabla L$
- ❖ Recalcular pérdida

Criterios de parada

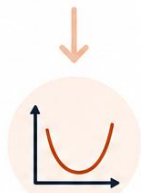
- ❖ Número fijo de épocas (ej. 500)
- ❖ Pérdida por debajo de umbral
- ❖ Early stopping: mejora insignificante

Backpropagation: la magia del aprendizaje



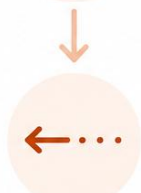
1 Forward pass

Datos fluyen hacia adelante:
 $X \rightarrow \text{capas} \rightarrow \hat{Y}$



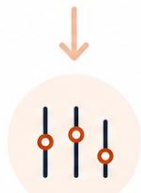
2 Calcular pérdida

$L = \text{MSE}(Y, \hat{Y})$ —
¿qué tan lejos estamos?



3 Backward pass

El error se propaga hacia atrás capa por capa usando la regla de la cadena



4 Actualizar pesos

Cada peso w se ajusta:
$$w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$$



Sin backpropagation, no podríamos entrenar **redes profundas**.



Calcula el gradiente de cada peso de forma **eficiente**.



Usa la **regla de la cadena** del cálculo diferencial.



Inventado en 1986; revolucionó el **deep learning**.



Backpropagation es el algoritmo que hace posible que una red neuronal **aprenda de sus errores**.

El gradiente guía cada actualización



Calcular pérdida

MSE entre Y predicho y Y real

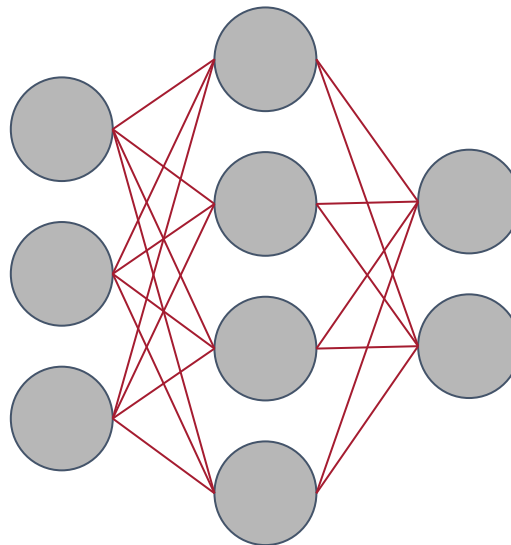
Calcular gradiente

$\partial L / \partial w$ para cada peso

Actualizar pesos

$$w = w - \alpha \cdot \nabla L$$

Donde α es la tasa de aprendizaje y ∇L es el gradiente de la pérdida.



Convergencia: el objetivo final



Época inicial

Pérdida alta, parámetros aleatorios

Desaceleración

Los ajustes se vuelven más pequeños

Descenso rápido

El gradiente reduce el error significativamente

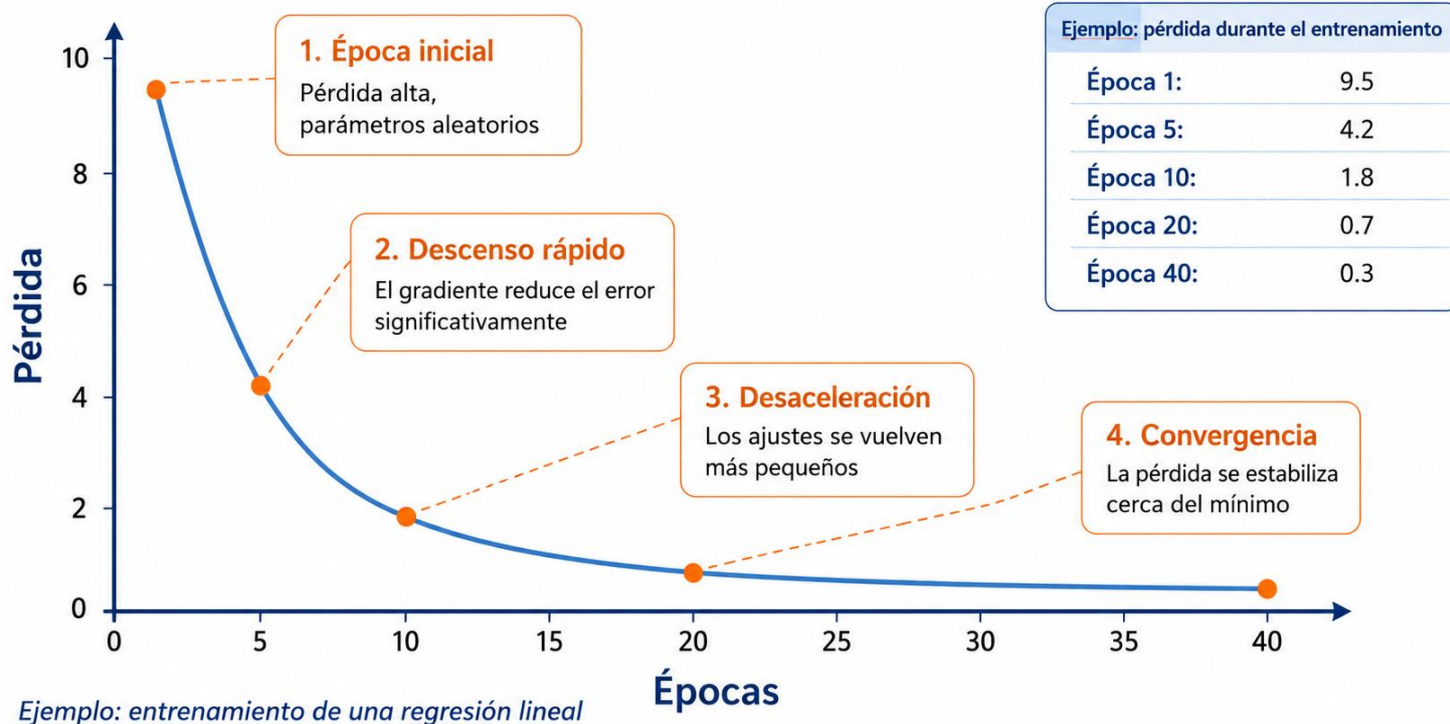
Convergencia

La pérdida se estabiliza en el mínimo



La curva de aprendizaje muestra cómo el modelo mejora progresivamente hasta estabilizarse.

Convergencia en el aprendizaje automático



Resumen: descenso de gradiente



Inicializar

Pesos w aleatorios

Forward pass

Calcular predicción $\hat{Y}$

Pérdida

$$\text{MSE} = (1/n)\sum (y - \hat{y})^2$$

Backprop

Gradientes $\partial L / \partial w$

Actualizar

$w \leftarrow w - \alpha \cdot \nabla L \rightarrow \text{repetir}$

SGD

Mini-lotes aleatorios. Rápido pero ruidoso.

Adam

Tasa adaptativa por parámetro. El más usado.

RMSProp

Escala según magnitud de gradientes recientes.



El descenso de gradiente es el motor de todo el aprendizaje automático moderno.

SGD: descenso de gradiente estocástico

Actualiza los pesos usando mini-lotes o muestras aleatorias.



1) Idea básica

En cada paso, el modelo calcula el gradiente con una muestra o mini-lote y actualiza los pesos para reducir la pérdida.

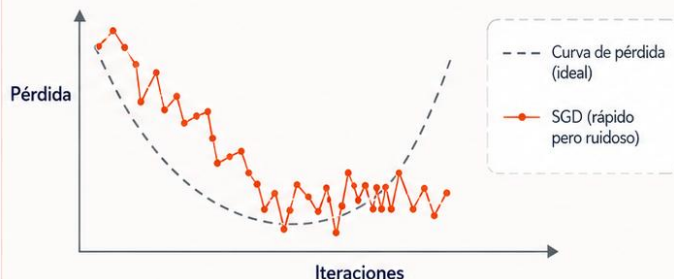
2) Ecuación de actualización

$$w_{t+1} = w_t - \alpha \nabla L$$

3) Cómo funciona (flujo)



4) Trayectoria del descenso con SGD



5) Ventajas

- Simple y eficiente
- Bajo costo computacional por paso
- Útil en grandes volúmenes de datos



6) Desventajas

- Trayectoria ruidosa
- Puede tardar en estabilizarse
- Sensible a la tasa de aprendizaje



7) Cuándo usarlo

Cuando se requiere un método sencillo y rápido para entrenar modelos con muchos datos.



Ejemplo: entrenamiento básico de una regresión o red neuronal.



Adam: optimización adaptativa

Combina momento y tasa de aprendizaje adaptativa para acelerar el entrenamiento



1) Idea básica

Adam ajusta cada parámetro usando promedios móviles del gradiente y de su magnitud, lo que produce actualizaciones más estables y rápidas.

2) Ecuaciones clave

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

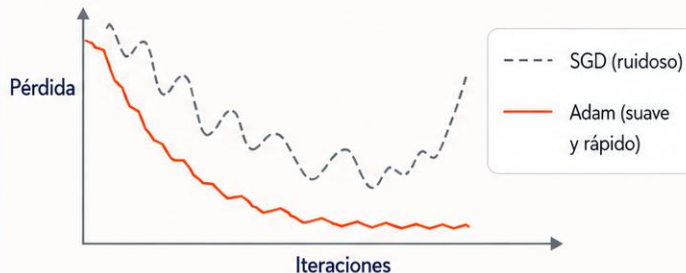
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$w_{t+1} = w_t - \alpha \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

3) Cómo funciona (flujo)



4) Trayectoria del descenso con Adam



5) Ventajas

- Convergencia rápida
- Actualización estable
- Muy usado en redes neuronales



6) Desventajas

- Más complejo que SGD
- Requiere más memoria
- Depende de hiperparámetros



7) Cuándo usarlo

Cuando se desea un entrenamiento robusto y eficiente en problemas de aprendizaje profundo.



Ejemplo: entrenamiento de redes neuronales profundas.



RMSProp: aprendizaje adaptativo

Escala la actualización según la magnitud reciente de los gradientes



1) Idea básica

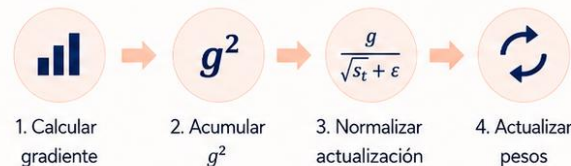
RMSProp mantiene un promedio de los gradientes al cuadrado para reducir oscilaciones y adaptar el tamaño de paso de cada parámetro.

2) Ecuaciones clave

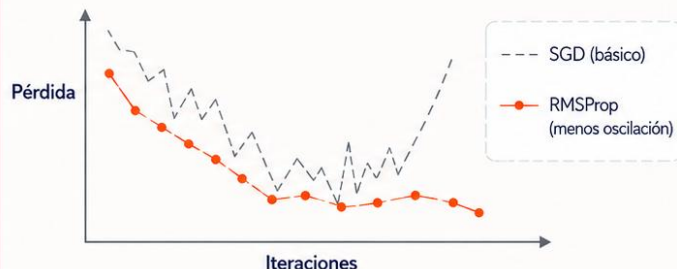
$$s_t = \rho s_{t-1} + (1 - \rho) g_t^2$$

$$w_{t+1} = w_t - \alpha \frac{g_t}{\sqrt{s_t} + \epsilon}$$

3) Cómo funciona (flujo)



4) Trayectoria del descenso con RMSProp



5) Ventajas

- Reduce oscilaciones
- Tasa adaptativa por parámetro
- Funciona bien en problemas no estacionarios



6) Desventajas

- Puede ser sensible a α
- Menos intuitivo que SGD
- No siempre generaliza mejor



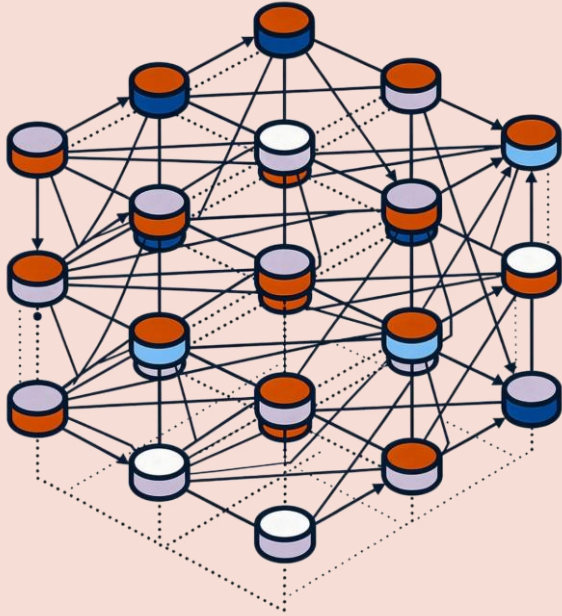
7) Cuándo usarlo

Quando se necesita estabilizar el aprendizaje y controlar gradientes de distinta magnitud.



Ejemplo: entrenamiento con gradientes variables o ruidosos.





Primera red neuronal

Uniendo todo con código

LAURENCE MORONEY, GOOGLE

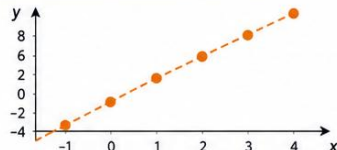
Ejemplo de una red neuronal simple

Una sola entrada, una sola neurona y una salida

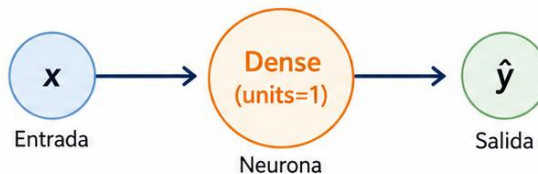
1. Datos de entrada

X	-1	0	1	2	3	4
Y	-3	-1	1	3	5	7

Los datos siguen aproximadamente la relación $y = 2x - 1$.



2. Modelo



La red aprende una relación lineal entre x e y .

3. Entrenamiento



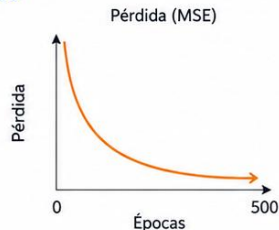
Optimizador: SGD



Función de pérdida: error cuadrático medio (MSE)



Épocas: 500



4. Resultado



Después del entrenamiento, el modelo aprende una recta similar a: $y \approx 2x - 1$

Ejemplo de predicción: $x = 10 \rightarrow \hat{y} \approx 19$



Esto significa que la red ajustó correctamente el patrón de los datos.

Este ejemplo muestra cómo una red neuronal sencilla puede aprender una relación lineal a partir de datos.

El código completo



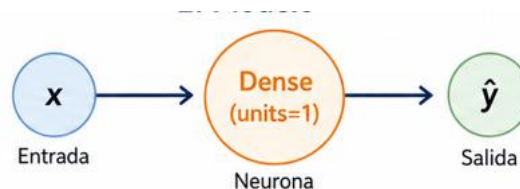
```
model = keras.Sequential([
    keras.layers.Dense(
        units=1,
        input_shape=[1])
])

model.compile(
    optimizer='sgd',
    loss='mean_squared_error'
)

xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0],
               dtype=float)
ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0],
               dtype=float)

model.fit(xs, ys, epochs=500)

print(model.predict([10.0]))
```



Anatomía del código

```
model = keras.Sequential([
    keras.layers.Dense(
        units=1,
        input_shape=[1])
])

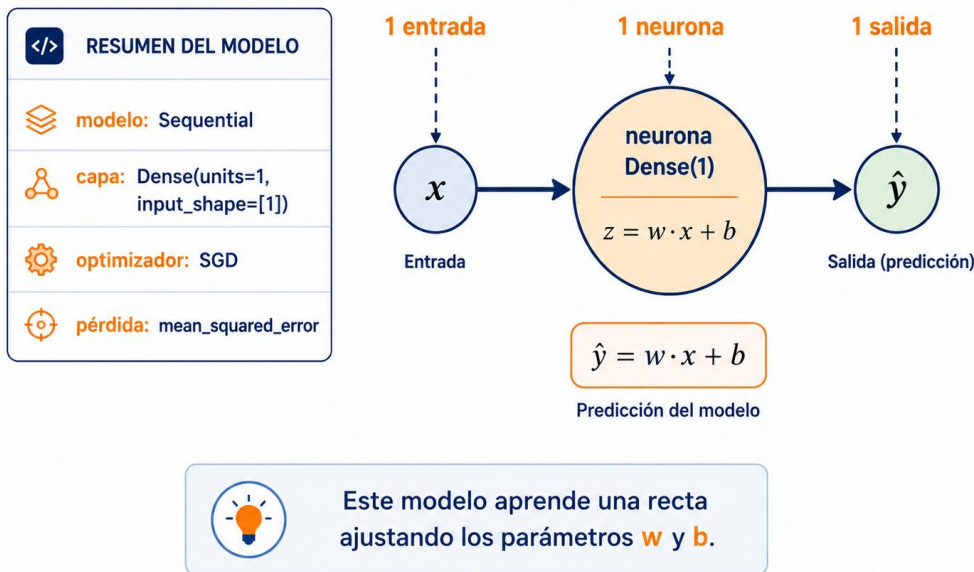
model.compile(
    optimizer='sgd',
    loss='mean_squared_error'
)

xs = np.array([-1.0, 0.0, 1.0, 2.0,
               3.0, 4.0], dtype=float)
ys = np.array([-3.0, -1.0, 1.0,
               3.0, 5.0, 7.0], dtype=float)

model.fit(xs, ys, epochs=500)

print(model.predict([10.0]))
```

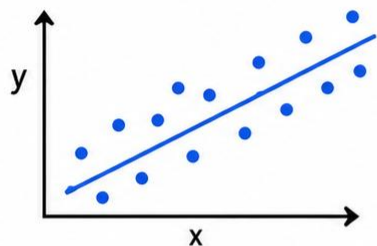
Red Neuronal: 1 entrada → 1 neurona → 1 salida



De regresión a clasificación

Cómo un modelo pasa de predecir valores a asignar clases

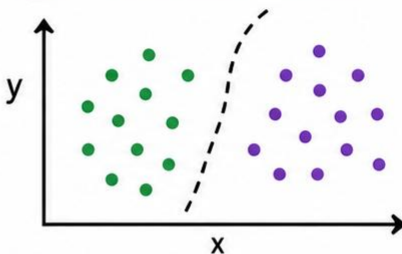
1 Regresión



→ Predice valores continuos

ej.: precio de una casa

2 Clasificación



→ Predice categorías discretas

ej.: spam / no spam

3 Diferencia clave

	Regresión (output)	= número real
	Clasificación (output)	= clase o probabilidad

4 Flujo general



Objetivo: que el modelo aprenda patrones y clasifique correctamente nuevos datos

Clasificación binaria

Cuando el modelo decide entre dos clases posibles

1 Idea básica



spam / no spam

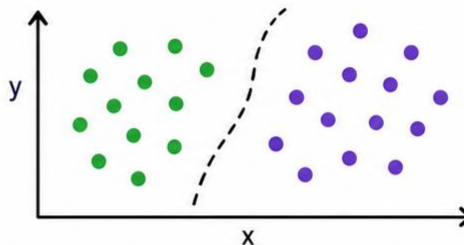


aprobado / no aprobado



la salida solo puede pertenecer a una de dos categorías.

2 Ejemplo visual



● clase 0

● clase 1

3 Ejemplo práctico

correo	etiqueta
¡Ganaste un premio!	spam
Reunión a las 10	no spam
Haz clic aquí	spam



modelo de clasificación



nuevo correo

predicción:
spam

4 Salida del modelo

spam 0.92

no spam 0.08

clase predicha:

spam

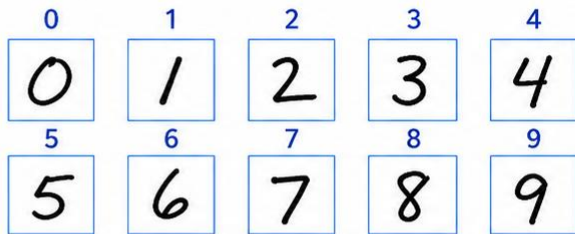


Idea clave: en clasificación binaria, el modelo elige una de dos clases a partir de patrones en los datos.

Clasificación multiclase

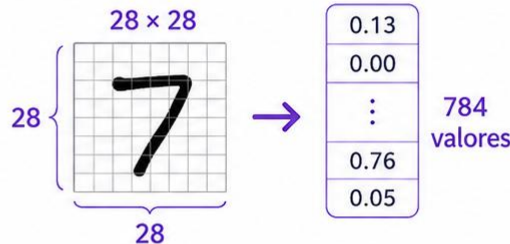
Cuando el modelo debe elegir entre varias categorías

1 Datos



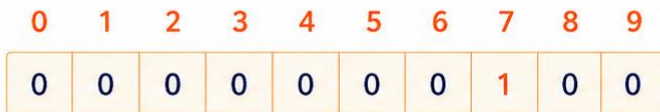
Cada imagen pertenece a una clase.

2 Representación



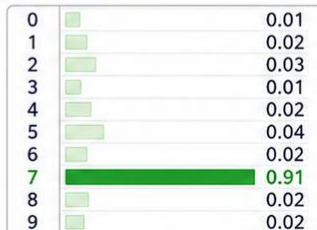
La imagen se convierte en números para entrar al modelo.

3 Etiquetas



Una clase activa, las demás en cero.

4 Salida



Clase predicha:

7

softmax

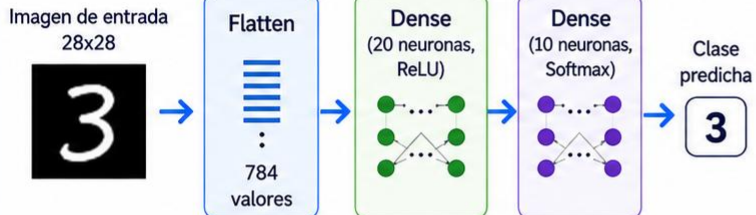


Idea clave: en clasificación multiclase, el modelo calcula probabilidades para varias clases y elige la mayor.

La red neuronal para clasificar

Un ejemplo sencillo con Keras y MNIST

1 Arquitectura



2 Código básico

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(20, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

3 Entrenamiento

Optimizador ●

Función de pérdida ●

Métricas ●

Épocas ●

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(x_train, y_train,
        validation_data=(x_test, y_test),
        epochs=20, batch_size=128)
```

4 Predicción

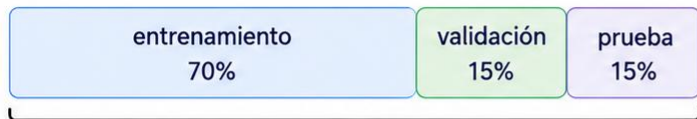


Este tipo de red aprende a reconocer **patrones visuales** y **asignar una clase** a cada imagen.

Entrenamiento, validación y prueba

Cómo evaluar si el modelo realmente aprende

1 División de datos



todo el conjunto de datos

→ Solo los datos de entrenamiento se usan para ajustar los pesos del modelo.

2 Durante el entrenamiento



datos de entrenamiento



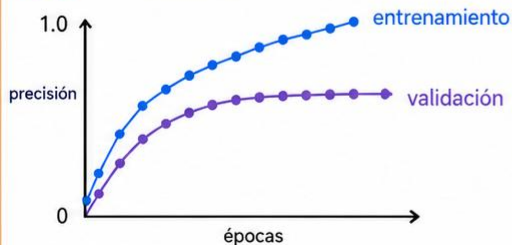
modelo



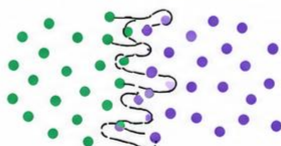
ajuste de pesos

→ La validación se usa para monitorear el desempeño con datos que el modelo no ha visto.

3 Precisión



4 Problema común



→ Si el modelo memoriza demasiado, puede ir muy bien en entrenamiento pero fallar con datos nuevos.

5 Conclusión



La prueba final estima el **desempeño real** del modelo.

● entrenamiento	0.99
● validación	0.92
● prueba	0.91



Idea clave: validar y probar evita conclusiones engañosas y ayuda a medir la generalización.

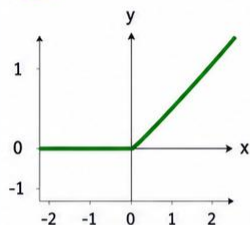
Funciones de activación en redes neuronales

Ejemplos gráficos, ecuaciones y uso recomendado



Las funciones de activación introducen no linealidad y ayudan a que la red aprenda patrones complejos.

1 ReLU



Ecuación:

$$f(x) = \max(0, x)$$



Uso recomendado:

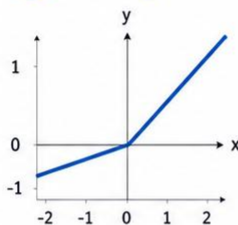
Capas ocultas en redes densas y CNN.



Nota:

Rápida y simple, pero puede sufrir neuronas muertas.

2 Leaky ReLU



Ecuación:

$$f(x) = \max(\alpha x, x), \text{ con } \alpha \text{ pequeño.}$$



Uso recomendado:

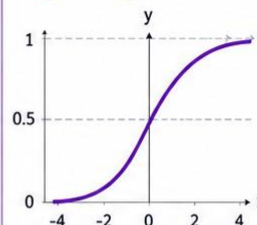
Alternativa a ReLU cuando hay riesgo de neuronas muertas.



Nota:

Mantiene un pequeño gradiente en valores negativos.

3 Sigmoid



Ecuación:

$$f(x) = \frac{1}{1 + e^{-x}}$$



Uso recomendado:

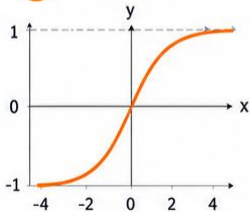
Salida para clasificación binaria y probabilidad de una clase.



Nota:

Interpretable como probabilidad, pero puede saturarse.

4 Tanh



Ecuación:

$$f(x) = \tanh(x)$$



Uso recomendado:

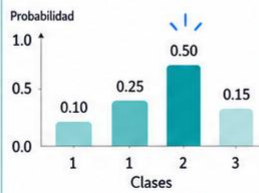
Capas recurrentes o cuando conviene salida centrada en cero.



Nota:

Centrada en cero, pero también puede saturarse.

5 Softmax



Suma = 1



Ecuación:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



Uso recomendado:

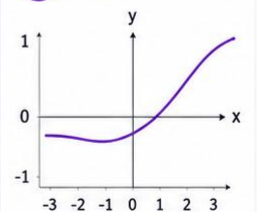
Capa de salida en clasificación multiclase.



Nota:

Convierte puntajes en probabilidades normalizadas.

6 GELU



Ecuación (aprox.):

$$\text{GELU}(x) \approx 0.5x \left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right) \right]$$



Uso recomendado:

Modelos modernos y redes profundas.



Nota:

Suave y efectiva, aunque más costosa que ReLU.



Conclusión: Elegir la función adecuada mejora el aprendizaje, la estabilidad y el rendimiento de tu red neuronal.

Guía rápida de uso y comparación de funciones de activación

Cuándo usar cada función y cómo compararlas

1 Guía rápida de uso: ¿qué función conviene y por qué?

1 Capas ocultas (MLP o CNN)

Conviene: **ReLU o Leaky ReLU**

¿Por qué?

- ✓ Sencillas y muy rápidas.
- ✓ Mitigan el problema del desvanecimiento del gradiente.
- ✓ Promueven activaciones dispersas (sparse) y entrenamiento estable.
- ✓ Leaky ReLU evita "neuronas muertas".

Regla práctica: ReLU por defecto. Si ves muchas neuronas muertas, usa Leaky ReLU.

2 Clasificación binaria

Conviene: **Sigmoid**

¿Por qué?

- ✓ Devuelve una probabilidad entre 0 y 1.
- ✓ Ideal para problemas de dos clases.
- ✓ Se usa en la capa de salida.

Úsala cuando tu salida sea: "¿Pertenece a la clase positiva?" Sí (1) o No (0).

3 Clasificación multiclase (k clases)

Conviene: **Softmax en la salida**

¿Por qué?

- ✓ Convierte puntajes en probabilidades que suman 1.
- ✓ Entrega una distribución sobre todas las clases.
- ✓ La clase con mayor probabilidad es la predicción.

Úsala cuando hay más de 2 clases excluyentes entre sí.

4 Modelos modernos / profundos

Conviene: **GELU como opción principal (Tanh en casos específicos)**

¿Por qué?

- ✓ GELU suele dar mejor desempeño en transformers y modelos grandes.
- ✓ Suaviza las activaciones y mejora la convergencia.
- ✓ Tanh útil en redes recurrentes (RNN) y salidas con rangos simétricos.

Si usas transformers → prueba GELU. Si tu salida debe estar en $[-1, 1]$ → Tanh.

2 Comparación rápida de funciones de activación

Criterio	ReLU	Leaky ReLU	Sigmoid	Tanh	Softmax	GELU
⚡ Velocidad de cálculo	Muy alta ★★★★★	Muy alta ★★★★★	Baja ★★★	Baja ★★★	Media ★★★	Media-Baja ★★★
🛡 Estabilidad en entrenamiento	Media ★★★	Alta ★★★★	Baja ★★★	Media ★★★	Alta ★★★★	Alta ★★★★
🎯 Salida / probabilidad	No acotada (0 a $+\infty$)	No acotada ($-\infty$ a $+\infty$)	Acotada (0 a 1) → Probabilidad	Acotada (-1 a 1) → Simétrica	Probabilidades que suman 1 (0 a 1)	No acotada ($-\infty$ a $+\infty$)
🔗 Uso típico	Capas ocultas en redes densas y CNN	Alternativa a ReLU cuando hay neuronas muertas	Salida binaria (clasificación 0/1)	RNN y salidas con rangos simétricos	Salida multiclase (clasificación k clases)	Transformers y modelos grandes / profundos
★ Principal ventaja	Muy rápida y simple; escala bien	Evita neuronas muertas; mejor fluidez del gradiente	Entrega probabilidad interpretable (0-1)	Centrada en 0; útil en RNN	Interpretable como distribución de clases	Mejor desempeño en modelos modernos; salida suave y estable



Conclusión:

No existe una función universal. La elección depende de la tarea, la arquitectura y los datos. Empieza con ReLU en capas ocultas y elige la función de salida adecuada para tu problema.



¿QUÉ VIENE AHORA? LA IA EN LA VIDA REAL

No se trata de un modelo mágico. Se trata de sistemas que empiezan a **trabajar**.



No será un solo modelo mágico que haga todo por sí mismo.
El salto es otra cosa.

LO QUE YA SE VE



MODELOS BASE MÁS FUERTES

Más capacidad, mejor razonamiento y mayor confiabilidad.



VENTANAS DE CONTEXTO MÁS GRANDES

Más información por conversación → mejores respuestas.



MEJOR INTEGRACIÓN CON HERRAMIENTAS

Conexión con APIs, datos, código y sistemas reales para ejecutar tareas.



SISTEMAS QUE SOSTIENEN TAREAS MÁS LARGAS

Planifican, ejecutan, verifican y corrigen con supervisión humana.

IMPACTO REAL, YA HOY



AlphaFold permitió predecir la estructura de prácticamente todas las proteínas conocidas, alrededor de **200 MILLONES**.



Reconocido con el **NOBEL DE QUÍMICA 2024**.



EL CAMBIO MÁS IMPORTANTE

Antes: "¿Puede parecer inteligente?"



Ahora:

"¿Qué tareas queremos delegarle?"

Con qué límites y con qué control humano.



SKILLS (CAPACIDADES)

- Son capacidades o funciones específicas que un sistema puede ejecutar.
- Aún no existe una definición única y estable en la industria.

En otro video explico mejor cómo se diferencia de MCP.



NO ES SOLO TECNOLOGÍA, ES UN DILEMA

- Qué delegar
- Qué no delegar
- Con qué límites
- Con qué control humano

Una decisión que definirá el impacto real de esta tecnología.

LO QUE VIENE: IA INTEGRADA EN LA VIDA COTIDIANA



MÁS ASISTENTES INTELIGENTES

Más Alexa, más Siri, más conversaciones naturales y útiles.



MÁS APPS Y SOFTWARE INTELIGENTE

IA integrada en las herramientas que usamos todos los días.



MÁS DISPOSITIVOS CONECTADOS

Hogares, autos, wearables y dispositivos cada vez más inteligentes.



MÁS ROBOTS Y AUTOMATIZACIÓN

Robots en entornos reales, ayudando en tareas físicas y digitales.



PARA MÁS PERSONAS

No solo para unos pocos. Tecnología cada vez más accesible y útil.

70 AÑOS DE IA, EN UNA SOLA IDEA



1) IMITAR

Intentamos que las máquinas imitaran.



2) REGLAS

Quisimos escribir la inteligencia en reglas.



3) APRENDER

Las hicimos aprender de datos.



4) GENERAR

Aprendieron a generar nuevo contenido.



5) ACTUAR



PARA CERRAR

Turing se preguntó si una máquina podía **parecer** humana.

Nuestra generación va a tener que decidir qué hacemos... cuando, además, **puede trabajar**.

Conclusiones



Paradigma ML

Datos + Labels $\rightarrow$ el modelo aprende las reglas

Función de pérdida

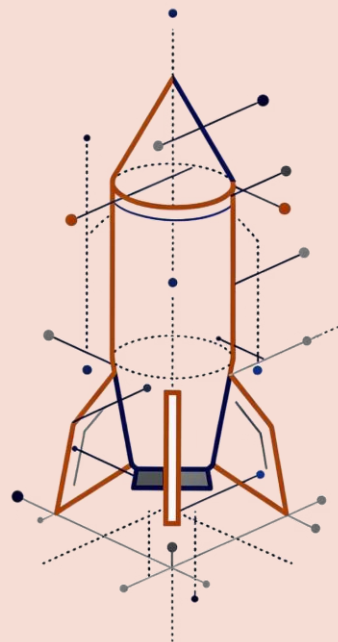
MSE mide qué tan lejos están las predicciones

Descenso de gradiente

$w \leftarrow w - \alpha \cdot \nabla L$ minimiza el error iterativamente

Red neuronal

10 líneas de Keras aprenden $Y = 2X - 1$



Conclusiones

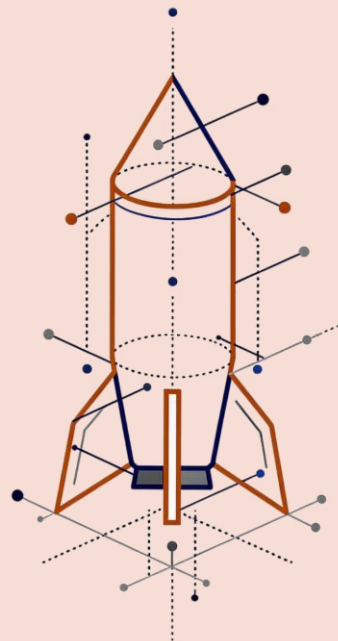


Conceptos clave

- ❖ **Labels:** respuestas conocidas para entrenar.
- ❖ **Época:** una pasada completa por los datos.
- ❖ **Backpropagation:** cómo fluye el error hacia atrás.
- ❖ Tasa de aprendizaje α : tamaño del paso de optimización.

¿Qué sigue?

- ❖ Redes neuronales más profundas (Deep Learning).
- ❖ Clasificación: más de dos categorías.
- ❖ Convoluciones para imágenes (CNN).
- ❖ Secuencias y lenguaje natural (RNN, Transformers).



Referencias



1. UNIFEI. *IESTI01 TinyML – Machine Learning for Embedding Devices* [Lista de reproducción]. YouTube.
<https://www.youtube.com/playlist?list=PLwmX8w17ZuqYg68Xg2Wgj3i89aehHJ0JV>
2. edX/Harvard. *Professional Certificate in Tiny Machine Learning (TinyML)*.
<https://www.edx.org/search?q=tiny-machine-learning>
3. Coursera/Edge Impulse. *Introduction to Embedded Machine Learning*.
<https://www.coursera.org/learn/introduction-to-embedded-machine-learning>
4. Coursera/Edge Impulse. *Computer Vision with Embedded Machine Learning*.
<https://www.coursera.org/learn/computer-vision-with-embedded-machine-learning>
5. Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.
<https://www.manning.com/books/deep-learning-with-python>
6. Warden, P., & Situnayake, D. (2019). *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media.
<https://www.oreilly.com/library/view/tinyml/9798341642423/>
7. Iodice, G. M. (2022). *TinyML Cookbook: Combining embedded, edge, and cloud machine learning for real-world solutions*. Packt Publishing.
Recuperado de <https://www.amazon.com/TinyML-Cookbook-learning-microcontrollers-real-world/dp/1837637369>
8. Situnayake, D., & Plunkett, J. (2022). *AI at the Edge: Solving real-world problems with embedded machine learning*. O'Reilly Media.
Recuperado de <https://www.oreilly.com/library/view/ai-at-the/9781098120191/>

Créditos y agradecimientos



Agradezco a los siguientes educadores e investigadores por compartir sus materiales de enseñanza:

- **Prof. Marcelo Rovai** – TinyML machine learning for embedding devices, UNIFEI
- **Prof. Vijay Janapa Reddi** – CS249r: Tiny machine learning, Harvard
- **Prof. Rahul Mangharam** – ESE3600: Tiny machine learning, University of Pennsylvania
- **Prof. Brian Plancher** – Harvard CS249r: Tiny machine learning, Barnard College, Columbia University
- **Shawn Hymel** – Edge Impulse
- **Pete Warden** – Google
- **Laurence Moroney** – Google
- **TinyMLedu Team** – TinyML Education Initiative

En el sitio web de TinyML4D se encuentran numerosos materiales educativos sobre TinyML. Todos son gratuitos y de código abierto con fines educativos; si utilizas el material, ¡por favor, cítalo! TinyML4D es una iniciativa para hacer que la educación en TinyML esté disponible para todos a nivel global.

Créditos:

Materiales educativos proporcionados por TinyML4D (<https://tinyml.seas.harvard.edu/showAndTell>), iniciativa internacional dedicada a democratizar el acceso al aprendizaje de TinyML en todo el mundo.



Declaración de uso de IA Generativa

- ❖ Este material fue elaborado con apoyo de herramientas de **inteligencia artificial generativa**.
- ❖ La IA se utilizó para apoyar la **organización y síntesis de contenidos**, redacción técnica, diseño de actividades, generación de ejemplos, figuras y recursos visuales.
- ❖ También se empleó como apoyo en la revisión de **código y materiales didácticos** relacionados con TinyML, Edge AI e instrumentación embebida.
- ❖ Todo el contenido fue **revisado, validado, seleccionado y adaptado por el instructor** antes de incorporarse.
- ❖ Las decisiones técnicas, la secuencia didáctica, los criterios de evaluación y la selección final de los recursos son responsabilidad del instructor.

Herramienta principal utilizada: ChatGPT (OpenAI).



La IAG es un apoyo para aprender mejor, no un atajo para dejar de pensar.